

SCHEMA GENERALE SOLUZIONE GREEDY

Ogni esercizio greedy richiede:

1. Formalizzazione del problema

- Definizione soluzione ammissibile
- Definizione costo/obiettivo

2. Scelta greedy - quale criterio locale?

3. Dimostrazione proprietà scelta greedy (Exchange Argument)

- Sia A^* soluzione ottima qualunque
- Sia a la scelta greedy
- Se $a \in A^* \rightarrow$ fine
- Altrimenti sostituisci elemento di A^* con a
- Dimostra che la nuova soluzione è ancora ottima

4. Dimostrazione sottostruttura ottima

- Dopo scelta greedy, il sottoproblema rimanente ha soluzione ottima

5. Pseudocodice

6. Analisi complessità

ESERCIZIO 1: SELEZIONE ATTIVITÀ (Activity Selection)

Problema

Date n attività con tempi di inizio $s[i]$ e fine $f[i]$, selezionare il massimo numero di attività mutuamente compatibili (non sovrapposte).

Due attività i e j sono **compatibili** se non si sovrappongono: $f[i] \leq s[j]$ o $f[j] \leq s[i]$.

Input:

- Array $s[1..n]$ tempi di inizio
- Array $f[1..n]$ tempi di fine

Output: Insieme A di indici di cardinalità massima tale che le attività siano mutuamente compatibili.

Scelta Greedy

Seleziona sempre l'attività che finisce prima tra quelle compatibili con le già selezionate.

Perché questa scelta? Finire prima lascia più tempo disponibile per le attività successive.

Dimostrazione Proprietà Scelta Greedy

Teorema: Esiste una soluzione ottima che contiene l'attività che finisce per prima.

Dimostrazione (Exchange Argument):

Sia $S = \{a_1, a_2, \dots, a_n\}$ insieme delle attività ordinate per tempo di fine: $f[1] \leq f[2] \leq \dots \leq f[n]$.

Sia a_1 l'attività che finisce per prima (scelta greedy).

Sia $A^* = \{a_{k_1}, a_{k_2}, \dots, a_{k_m}\}$ una soluzione ottima qualunque, con $k_1 < k_2 < \dots < k_m$.

Caso 1: Se $a_{k_1} = a_1$, abbiamo finito (la scelta greedy è già in A^*).

Caso 2: Se $a_{k_1} \neq a_1$, allora $k_1 > 1$, quindi $f[1] \leq f[k_1]$ per l'ordinamento.

Costruiamo $A' = (A^* \setminus \{a_{k_1}\}) \cup \{a_1\}$ sostituendo a_{k_1} con a_1 .

Verifichiamo che A' è soluzione ammissibile:

- a_1 è compatibile con a_{k_2} perché $f[1] \leq f[k_1] \leq s[k_2]$ (a_{k_1} era compatibile con a_{k_2})
- a_1 è compatibile con tutte le altre attività in A^* per lo stesso motivo

Verifichiamo che A' è ottima:

- $|A'| = |A^*|$ (abbiamo solo sostituito un'attività)
- Quindi A' è ottima e contiene la scelta greedy a_1

Conclusione: Esiste sempre una soluzione ottima che contiene la scelta greedy. ■

Dimostrazione Sottostruttura Ottima

Teorema: Dopo aver scelto a_1 , il sottoproblema delle attività compatibili con a_1 ha sottostruttura ottima.

Dimostrazione:

Sia $S' = \{a_i \in S : s[i] \geq f[1]\}$ il sottoproblema (attività che iniziano dopo la fine di a_1).

Sia A soluzione ottima che contiene a_1 . Allora $A \setminus \{a_1\}$ è soluzione ottima per S' .

Per assurdo: Se esistesse A'' soluzione per S' con $|A''| > |A \setminus \{a_1\}|$, allora $A'' \cup \{a_1\}$ sarebbe soluzione per S con cardinalità $|A''| + 1 > |A|$, contraddicendo l'ottimalità di A^* . ■

Pseudocodice

```
ActivitySelection(s, f, n):  
    // PREREQUISITO: attività ordinate per f[i] crescente  
    A = {1}           // Prima attività (finisce prima)  
    last = 1
```

```

for i = 2 to n:
    if s[i] ≥ f[last]:    // Compatibile con ultima scelta
        A = A ∪ {i}
        last = i

return A

```

Analisi Complessità

- **Ordinamento:** $O(n \log n)$ se non già ordinato
- **Selezione:** $O(n)$ un solo passaggio
- **Totale:** $O(n \log n)$

Esempio Completo

Input:

```

i:   1  2  3  4  5  6
s = [2, 3, 5, 2, 1, 9]
f = [4, 7, 8, 9, 10, 11]

```

Passo 1: Ordina per $f[i]$:

```

i:   1  2  3  5  4  6
s = [2, 3, 5, 1, 2, 9]
f = [4, 7, 8, 10, 9, 11]

```

Passo 2: Selezione greedy:

- Scegli a_1 (finisce a 4)
- a_2 : $s[2]=3 < f[1]=4 \rightarrow$ incompatibile, salta
- a_3 : $s[3]=5 \geq f[1]=4 \rightarrow$ compatibile, scegli (finisce a 8)
- a_5 : $s[5]=1 < f[3]=8 \rightarrow$ incompatibile, salta
- a_4 : $s[4]=2 < f[3]=8 \rightarrow$ incompatibile, salta
- a_6 : $s[6]=9 \geq f[3]=8 \rightarrow$ compatibile, scegli

Output: $A = \{1, 3, 6\}$

ESERCIZIO 2: PROBLEMA DEI TURNI (Interval Scheduling)

Problema

Dato n richieste temporali $r[1..n]$ (istanti di tempo), trovare il numero minimo di turni di durata fissa per coprire tutte le richieste.

Ogni turno dura 1 ora e copre intervallo $[t, t+1)$ a partire dall'istante t .

Input: Array $r[1..n]$ di istanti di richiesta

Output: Sequenza $t[1..k]$ di istanti di inizio turni, k minimo, tale che ogni $r[i]$ è coperto da qualche turno.

Formalizzazione

Soluzione ammissibile: Sequenza $t[1..k]$ tale che $\forall i: \exists j$ con $r[i] \in [t[j], t[j]+1)$

Costo: k (numero di turni)

Obiettivo: Minimizzare k

Scelta Greedy

Assumendo r ordinato crescentemente:

1. Primo turno: $t[1] = r[1]$
2. Per ogni richiesta successiva:
 - Se coperta dal turno corrente $\rightarrow$ continua
 - Altrimenti $\rightarrow$ apri nuovo turno

Criterio: Apri nuovo turno solo quando strettamente necessario.

Dimostrazione Proprietà Scelta Greedy

Teorema: Esiste una soluzione ottima che inizia il primo turno a $r[1]$.

Dimostrazione:

Sia $r[1..n]$ ordinato: $r[1] \leq r[2] \leq \dots \leq r[n]$.

Sia $T^* = \{t'_1, t'_2, \dots, t'_m\}$ soluzione ottima con turni ordinati: $t'_1 \leq t'_2 \leq \dots \leq t'_m$.

Poiché $r[1]$ deve essere coperto: $r[1] \in [t'_1, t'_1+1)$, quindi $t'_1 \leq r[1] < t'_1+1$.

Costruiamo T' sostituendo t'_1 con $t_1 = r[1]$:

$T' = \{r[1], t'_2, \dots, t'_m\}$

Verifichiamo che T' copre tutte le richieste:

- Tutte le richieste coperte da t'_1 sono anche coperte da $r[1]$ perché:
 - Se $r[i] \in [t'_1, t'_1+1)$ allora $t'_1 \leq r[i] < t'_1+1$
 - Poiché $t'_1 \leq r[1] \leq r[i]$ (ordinamento) e $r[i] < t'_1+1 \leq r[1]+1$

- Quindi $r[i] \in [r[1], r[1]+1)$ ✓

Conclusione: T è ottima (stesso numero di turni) e contiene la scelta greedy. ■

Dimostrazione Sottostruttura Ottima

Dopo aver scelto $t[1] = r[1]$, il sottoproblema consiste nelle richieste $r[i] > r[1]+1$.

Se T è ottima per il problema originale e contiene $t[1]$, allora $T \setminus \{t[1]\}$ è ottima per il sottoproblema. (Dimostrazione per assurdo come Activity Selection.)

Pseudocodice

```
MinimumTurns(r, n):
    // PREREQUISITO: r ordinato crescente
    t[1] = r[1]
    turns = 1

    for i = 2 to n:
        if r[i] > t[turns] + 1:    // Non coperto dal turno corrente
            turns = turns + 1
            t[turns] = r[i]

    return (t, turns)
```

Analisi Complessità

- **Ordinamento:** $O(n \log n)$
- **Selezione:** $O(n)$
- **Totale:** $O(n \log n)$

Esempio

Input: $r = [1, 2, 3, 7, 8, 9]$

Esecuzione:

- $t[1] = 1$ (copre $[1, 2)$)
- $r[2] = 2$: $2 > 1+1$? No → coperto
- $r[3] = 3$: $3 > 1+1$? Sì → $t[2] = 3$ (copre $[3, 4)$)
- $r[4] = 7$: $7 > 3+1$? Sì → $t[3] = 7$ (copre $[7, 8)$)
- $r[5] = 8$: $8 > 7+1$? No → coperto
- $r[6] = 9$: $9 > 7+1$? Sì → $t[4] = 9$

Output: 4 turni: $[1, 2)$, $[3, 4)$, $[7, 8)$, $[9, 10)$



ESERCIZIO 3: COPERTURA INTERVALLI UNITARI

Problema

Dato insieme $X = \{x_1, x_2, \dots, x_n\}$ di punti ordinati sulla retta reale, trovare insieme I di cardinalità minima di intervalli chiusi di ampiezza 1 tale che ogni punto di X sia coperto da almeno un intervallo.

Input: $X[1..n]$ ordinato: $x[1] \leq x[2] \leq \dots \leq x[n]$

Output: Insieme $I = \{[a_1, a_1+1], [a_2, a_2+1], \dots, [a_k, a_k+1]\}$ di cardinalità minima tale che $\forall x_i \in X, \exists j: x_i \in [a_j, a_j+1]$

Scelta Greedy

Primo intervallo: $[x[1], x[1]+1]$

Criterio: Inizia l'intervallo dal primo punto non ancora coperto.

Dimostrazione Proprietà Scelta Greedy

Teorema: Esiste soluzione ottima che contiene l'intervallo $[x[1], x[1]+1]$.

Dimostrazione:

Sia I *soluzione ottima*, $[a, a+1] \in I$ intervallo che copre $x[1]$.

Quindi $a \leq x[1] \leq a+1$.

Caso 1: $a = x[1] \rightarrow$ intervallo coincide con scelta greedy, fine.

Caso 2: $a < x[1] \rightarrow a \leq x[1] \leq a+1$

Tutti i punti coperti da $[a, a+1]$ sono anche coperti da $[x[1], x[1]+1]$:

- Se $x_j \in [a, a+1]$ allora $a \leq x_j \leq a+1$
- Poiché $a < x[1] \leq x_j$ (ordinamento) e $x_j \leq a+1 < x[1]+1$
- Quindi $x_j \in [x[1], x[1]+1] \checkmark$

Sostituendo $[a, a+1]$ con $[x[1], x[1]+1]$ otteniamo I' con $|I'| = |I|$ che copre tutti i punti.

Conclusione: I' è ottima e contiene scelta greedy. ■

Pseudocodice

```
MinCover(X, n):  
    // PREREQUISITO: X ordinato crescente  
    I = {[X[1], X[1]+1]}  
    last = 1
```

```

for i = 2 to n:
    if X[i] > X[last] + 1:      // Non coperto
        I = I ∪ {[X[i], X[i]+1]}
        last = i

return I

```

Analisi Complessità

$O(n)$ se già ordinato, $O(n \log n)$ altrimenti.

Esempio

Input: $X = [1, 1.5, 2, 4, 4.2, 4.8, 7]$

Esecuzione:

- $I = \{[1, 2]\}$ copre $\{1, 1.5, 2\}$
- $x[4] = 4 > 2$? Sì $\rightarrow I = \{[1, 2], [4, 5]\}$ copre $\{4, 4.2, 4.8\}$
- $x[7] = 7 > 5$? Sì $\rightarrow I = \{[1, 2], [4, 5], [7, 8]\}$

Output: 3 intervalli

ESERCIZIO 4: SCHEDULING PROGRAMMI (Minimizzare Tempo Completamento)

Problema

Abbiamo n programmi da eseguire sul computer. Programma j ha lunghezza l_j (tempo esecuzione).

Dato ordine esecuzione $\sigma = (j_1, j_2, \dots, j_n)$, il tempo di completamento $C_{ji}(\sigma)$ del programma j_i è:

$$C_{ji}(\sigma) = \sum_{k=1}^i l_{j_k}$$

Obiettivo: Trovare ordine σ^* che minimizza $\sum_{i=1}^n C_{ji}(\sigma)$

Scelta Greedy

Ordina i programmi per lunghezza crescente: $l_1 \leq l_2 \leq \dots \leq l_n$

Esegui nell'ordine: $(1, 2, \dots, n)$

Dimostrazione Proprietà Scelta Greedy

Lemma: Se $l_i < l_j$, scambiare due programmi adiacenti i, j nell'ordine (con i dopo j) riduce il costo totale.

Dimostrazione Lemma:

Consideriamo sequenza: $\dots \rightarrow j \rightarrow i \rightarrow \dots$

Tempi completamento:

- $C_j = T + l_j$ (dove T è tempo precedente)
- $C_i = T + l_j + l_i$
- $\text{Somma} = 2T + 2l_j + l_i$

Scambiando: $\dots \rightarrow i \rightarrow j \rightarrow \dots$

Tempi completamento:

- $C'_i = T + l_i$
- $C'_j = T + l_i + l_j$
- $\text{Somma}' = 2T + 2l_i + l_j$

Confronto: $\text{Somma} - \text{Somma}' = (2T + 2l_j + l_i) - (2T + 2l_i + l_j) = l_j - l_i > 0$

Quindi $\text{Somma}' < \text{Somma}$. ✓

Teorema Principale: L'ordinamento crescente è ottimo.

Dimostrazione:

Sia σ *ordinamento ottimo*. Se σ non è crescente, esistono $i < j$ con $l_i > l_j$.

Scambiando i e j otteniamo σ' con costo minore (per il Lemma), contraddicendo l'ottimalità di σ^* .

Quindi σ^* deve essere crescente. ■

Pseudocodice

```
SchedulePrograms(l, n):  
    Sort(l)           // Ordina per lunghezza  
    return (1, 2, ..., n)
```

Complessità

$O(n \log n)$ per ordinamento

Esempio

Input: $l = [5, 2, 8, 3]$

Ordinamento: $[2, 3, 5, 8]$

Ordine ottimo: j_2, j_4, j_1, j_3

Tempi completamento:

- $C_2 = 2$
- $C_4 = 2+3 = 5$
- $C_1 = 2+3+5 = 10$
- $C_3 = 2+3+5+8 = 18$

Somma: $2+5+10+18 = 35$



ESERCIZIO 5: HUFFMAN CODING

Problema

Dato alfabeto $\{a_1, \dots, a_n\}$ con frequenze $\{f_1, \dots, f_n\}$, costruire codice prefisso con lunghezza media minima.

Lunghezza media: $\sum_i f_i \cdot l_i$ dove l_i è lunghezza codeword per a_i .

Scelta Greedy

Unisci i due simboli con frequenza minima creando nodo interno con frequenza = somma.

Ripeti finché rimane un solo albero.

Pseudocodice

```
Huffman(C):  
    n = |C|  
    Q = MinHeap(C)      // Priority queue su frequenze  
  
    for i = 1 to n-1:  
        z = new Node()  
        z.left = ExtractMin(Q)  
        z.right = ExtractMin(Q)  
        z.freq = z.left.freq + z.right.freq  
        Insert(Q, z)  
  
    return ExtractMin(Q)  // Radice albero
```

Complessità

$O(n \log n)$ - n inserimenti/estrazioni da heap

Esempio

Input:

Simbolo:	A	B	C	D	E
Freq:	12%	11%	53%	10%	14%

Costruzione:

1. Unisci D(10%) + B(11%) → DB(21%)
2. Unisci A(12%) + E(14%) → AE(26%)
3. Unisci DB(21%) + AE(26%) → DBAE(47%)
4. Unisci C(53%) + DBAE(47%) → Radice(100%)

Codici risultanti:

C: 0
A: 110
E: 111
D: 100
B: 101

Lunghezza media: $0.53 \cdot 1 + 0.12 \cdot 3 + 0.14 \cdot 3 + 0.10 \cdot 3 + 0.11 \cdot 3 = 1.61$ bit

TEMPLATE GENERALE PER ESAME

PROBLEMA: [descrizione]

FORMALIZZAZIONE:

- Soluzione ammissibile: [definizione]
- Costo: [funzione obiettivo]
- Obiettivo: [massimizzare/minimizzare]

SCELTA GREEDY: [criterio locale]

DIMOSTRAZIONE PROPRIETÀ SCELTA GREEDY:

Teorema: Esiste soluzione ottima che contiene la scelta greedy.

Dimostrazione (Exchange Argument):

Sia A^* soluzione ottima qualunque

Sia a scelta greedy

[Se $a \in A^*$, fine]

[Altrimenti, sostituisci elemento di A^* con a]

[Verifica ammissibilità e ottimalità]

Conclusione: ... ■

DIMOSTRAZIONE SOTTOSTRUTTURA OTTIMA:

Teorema: Dopo scelta greedy, sottoproblema ha soluzione ottima.

Dimostrazione: [per assurdo o diretta]

PSEUDOCODICE:

[algoritmo]

COMPLESSITÀ: [analisi]



CONSIGLI PER L'ESAME

Quando Greedy Funziona

✓ Ordinamento naturale dei dati ✓ Scelta locale ovvia ✓ Exchange argument dimostrabile ✓
Problema "flusso temporale"

Quando NON Funziona

X Coin Change con {1, 20, 50} X Problemi con dipendenze complesse X Knapsack 0/1

Errori da Evitare

✗ Non dimostrare proprietà scelta greedy ✗ Assumere greedy ottimo senza dimostrazione ✗ Confondere "funziona" con "è ottimo" ✗ Non verificare sottostruttura ottima

Checklist Soluzione

- ☐ Formalizzazione completa
- ☐ Scelta greedy esplicitata
- ☐ Exchange argument completo
- ☐ Sottostruttura ottima dimostrata
- ☐ Pseudocodice chiaro
- ☐ Complessità analizzata